

Reviewer Pack — Minimal Rank of Configuration Spaces over Function Fields vs...

Entry 288b1eeba389 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 18:40:20 UTC

Minimal Rank of Configuration Spaces over Function Fields vs Communication Complexity for k-Clique

Entry ID: 288b1eeba389 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 18:40:20 UTC

1. Conjecture

Field A (mathematical branch): Topological K-theory (Configuration Spaces) **Field B** (complexity object): Communication Complexity (k-Clique)

Statement:

[‘For a given function field $F_q(T)$ and a k -clique instance on n vertices, the minimal rank of the configuration space $CS(F_q(T))$ associated with the instance is lower-bounded by $\Omega(\log q^{(n/2)})$.’, ‘Where, $CS(F_q(T))$ is defined as the set of all configurations of points in $F_q(T)$ such that the distance between any two points is at least 1.’, ‘This implies a direct correspondence between the complexity of solving k -clique and the topological properties of configuration spaces over function fields.’]

Rationale (proposer’s reasoning):

[‘Configuration spaces have been previously studied in the context of geometric and algebraic topology, but their relationship to communication complexity remains largely unexplored.’, ‘This conjecture suggests that the complexity of certain problems, such as k -clique, can be understood through topological concepts, potentially leading to new insights into both fields.’]

Taxonomy category: Topological_K_theory_Configuration_Spaces (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): baaf57c6727d1363

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all n from 1 to 40, the computed minimal rank of $CS(F_q(T))$ associated with a k -clique instance on n vertices meets the lower bound $\Omega(\log q^{(n/2)})$, and this condition holds true across at least 80% of the 30 random seeds. The conjecture is falsified if for any n , the computed minimal rank does not meet the lower bound or this condition fails to hold in less than 80% of the seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        pivot_row = i
        for j in range(i + 1, rows):
            if abs(matrix[j][i]) > abs(matrix[pivot_row][i]):
                pivot_row = j
        matrix[i], matrix[pivot_row] = matrix[pivot_row], matrix[i]
        pivot = matrix[i][i]
        if pivot == 0:
            raise ValueError("Matrix is singular and cannot be reduced.")
        for j in range(i, cols):
            matrix[i][j] /= pivot
        for k in range(rows):
            if k != i:
                factor = matrix[k][i]
                for j in range(i, cols):
                    matrix[k][j] -= factor * matrix[i][j]
    return matrix

def minimal_rank(n, q):
    # Construct a random k-clique instance on n vertices
    edges = set()
    while len(edges) < n:
        u, v = random.sample(range(n), 2)
        if u != v and (u, v) not in edges and (v, u) not in edges:
            edges.add((u, v))

    # Map the k-clique to a set of points in F_q(T)
    points = []
```

```

for i in range(n):
    point = [random.randint(0, q - 1) for _ in range(n)]
    points.append(point)

# Create the distance matrix
distance_matrix = [[0] * n for _ in range(n)]
for i in range(n):
    for j in range(i + 1, n):
        dist = sum((points[i][k] - points[j][k]) ** 2 for k in range(n))
        if dist == 0:
            raise ValueError("Points are not distinct.")
        distance_matrix[i][j] = dist
        distance_matrix[j][i] = dist

# Create the configuration space matrix
config_space_matrix = []
for i in range(n):
    row = [1]
    for j in range(i + 1, n):
        if (i, j) in edges:
            row.append(distance_matrix[i][j])
        else:
            row.append(0)
    config_space_matrix.append(row)

# Compute the minimal rank
reduced_matrix = gaussian_elimination(config_space_matrix)
rank = sum(1 for row in reduced_matrix if any(val != 0 for val in row))
return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    q = random.randint(2, 10)
    n_values = [5, 10, 15, 20, 30, 40]
    metric_name = "minimal_rank"
    total_metric_value = 0
    instances_tested = 0

    for n in n_values:
        rank = minimal_rank(n, q)
        expected_bound = Fraction(1, 2) * n * (n - 1) * math.log(q**(n/2))
        if rank < expected_bound:
            return {
                "metric_name": metric_name,
                "metric_value": rank,
                "instances_tested": instances_tested + 1,
                "conjecture_holds": False,
                "counterexample": f"Rank {rank} is less than the expected bound {expected_bound}"
            }
        total_metric_value += rank
        instances_tested += 1

    mean_metric_value = total_metric_value / instances_tested
    return {
        "metric_name": metric_name,
        "metric_value": mean_metric_value,

```

```

        "instances_tested": instances_tested,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Rank below expected bound\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f0ec242b.py", line 115, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f0ec242b.py", line 87, in run_trial
    rank = minimal_rank(n, q)
           ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f0ec242b.py", line 74, in minimal_rank
    reduced_matrix = gaussian_elimination(config_space_matrix)
                     ~~~~~^~~~~~

  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f0ec242b.py", line 35, in gaussian_elimination
    matrix[k][j] -= factor * matrix[i][j]
    ~~~~~^~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that it was unable to complete the required computations and verify the conjecture. | next: Re-run the test with a different seed or investigate the cause of the crash to ensure that the computation can be completed successfully.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11444
2	preregistration	ollama_remote	glm4:latest	0	0	6092
3	novelty	ollama_remote	glm4:latest	0	0	4913
4	novelty	ollama_remote	glm4:latest	0	0	5160
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14163
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13305
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8500
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13030
9	judge	ollama_remote	glm4:latest	0	0	12335

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 88944 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/288bleeba389.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/288bleeba389.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/288bleeba389.tar.gz (if generated)